Informe Técnico

🎯 Propósito del documento

Es un *Technical Design & Development Report* (TDDR) que reúne el diseño, el desarrollo y la validación del Paciente Virtual Adaptativo, con la evidencia de los experimentos y del prototipo.

📐 ESTRUCTURA DEL DOCUMENTO

SECCIÓN 0 — PORTADA Y METADATOS DEL PROYECTO

Paciente Virtual Adaptativo es una aplicación web de simulación clínica, con un núcleo determinístico y una capa de IA generativa que solo redacta.

Campo	Descripción
Tipo de solución	Híbrida: aplicación web con IA generativa (agentes con modelo de lenguaje) sobre un núcleo de reglas determinístico
Dominio de aplicación	Salud, educación médica: simulación clínica para entrenar el razonamiento diagnóstico de estudiantes de medicina
Repositorio del código	https://github.com/JamesGuti3rrez/consulta-medica.git
Dataset	MedQA (USMLE, 4 opciones) para el contenido clínico de los casos de referencia y casos sintéticos diseñados por el equipo en familias de diagnóstico diferencial. En evaluación: CasiMedicos / MedExpQA, casos del examen MIR en español.

SECCIÓN 1 — PROBLEMA Y MOTIVACIÓN TÉCNICA

1.1 Descripción del problema real

- Contexto del dominio donde existe el problema

Los estudiantes de medicina aprenden sobre todo con casos ya resueltos: leen el caso y su diagnóstico juntos. Casi nunca entrenan el proceso de llegar al diagnóstico: preguntar lo correcto en la anamnesis, revisar los datos disponibles, plantear hipótesis y descartarlas con evidencia. Practicar ese proceso con pacientes reales es limitado y conlleva riesgo para el paciente.

- Evidencia cuantitativa del problema (estadísticas, reportes, datos reales)

Indicador	Valor	Fuente
Personas con muerte o discapacidad permanente por error diagnóstico, por año, en EE. UU.	795 000 (rango plausible 598 000 a 1 023 000)	Newman-Toker et al., BMJ Qual Saf, 2024
Tasa media de error diagnóstico en 15 enfermedades peligrosas (vasculares, infecciones, cáncer)	11,1 %	Newman-Toker et al., BMJ Qual Saf, 2024
Casos de error diagnóstico en medicina interna con factores cognitivos	74 % de 100 casos; el cierre prematuro fue la causa más común	Graber et al., Arch Intern Med, 2005

Efecto de los pacientes virtuales sobre las habilidades frente a la enseñanza tradicional	SMD 0,90 (IC 95 % 0,49 a 1,32), 51 ensayos y 4 696 participantes	Kononowicz et al., J Med Internet Res, 2019
Variación de exactitud de un modelo de lenguaje entre corridas idénticas con temperatura 0	Hasta 15 %	Atil et al., 2024

- ¿Por qué las soluciones actuales son insuficientes?
  - Casos resueltos y libros: entregan el diagnóstico junto con el caso, así que no entrenan el proceso de llegar a él.
  - Pacientes estandarizados (actores): requieren actores, espacio y docentes por cada sesión, lo que limita cuántas veces practica cada estudiante.
  - Pacientes virtuales con guion fijo: responden siempre con el mismo texto y solo a las preguntas previstas. El propio prototipo del equipo empezó así (`/api/status` informaba `scripted: true`).
  - Chatbots con un modelo de lenguaje sin control: no son deterministas y pueden inventar datos o revelar el diagnóstico. En los experimentos del equipo, con una instrucción simple la IA no produjo ningún caso válido (0 de 60), y con el contexto compartido el diagnóstico se filtró en 3 de 4 ataques.

1.2 Brecha tecnológica identificada

- ¿Qué gap técnico específico aborda esta solución?

Gap técnico. Las soluciones existentes eligen entre dos extremos: conversación natural sin control (chatbot con modelo de lenguaje) o control total sin conversación natural (guion fijo). Falta una arquitectura que combine ambas cosas:

- Conversación en lenguaje natural, por voz, con varios roles (médico asistente, paciente, laboratorio, radiología, especialista).
- Revelación progresiva y controlada de cada dato clínico, según lo que el estudiante pide.
- Garantía de que el modelo nunca ve el diagnóstico ni inventa datos.
- Evaluación determinística del proceso, no solo del resultado, con errores clasificados por tipo.

- Justificación de por qué se necesita una solución computacional

Hace falta generar lenguaje natural en tiempo real, llevar el estado de lo revelado en cada turno, aplicar reglas clínicas de forma repetible y registrar cada paso para explicarlo después. Una solución computacional permite además que cualquier estudiante practique sin un docente presente y tantas veces como necesite.

1.3 Pregunta de investigación técnica

- Formulada como: ¿En qué medida un simulador clínico web que combina agentes conversacionales con modelo de lenguaje y un núcleo de reglas determinístico permite entrenar y evaluar el razonamiento diagnóstico, sin revelar el diagnóstico ni inventar datos clínicos, en el contexto de la formación de estudiantes de medicina?

1.4 Objetivo general y objetivos específicos

- Objetivo General: Desarrollar un simulador clínico web en el que el estudiante entrevista por voz a un paciente virtual, solicita exámenes, plantea y descarta hipótesis y recibe una evaluación de su proceso, con la lógica clínica y la evaluación separadas del modelo de lenguaje.
- Objetivos específicos medibles:

N.º	Objetivo específico	Indicador medible	Sección

OE1	Construir un repositorio de casos clínicos canónicos con esquema JSON validado.	100 % de los casos publicados pasan el validador y la curación clínica.	3.3
OE2	Implementar la orquestación de agentes con acceso a los datos por rol.	0 fugas diagnósticas y 0 fugas entre roles en la batería de ataques; routing correcto en 100 % de los turnos.	3.1 y 3.5
OE3	Implementar el motor de reglas (alergias, contraindicaciones, revelación) y la evaluación con errores clasificados.	La misma entrada produce el mismo resultado en el 100 % de las repeticiones.	3.2
OE4	Desarrollar la interfaz web accesible con interacción por voz.	Recorrido completo terminado; 0 violaciones críticas en axe-core; Lighthouse de accesibilidad ≥ 90.	3.3
OE5	Proteger las sesiones y los datos de los usuarios.	0 accesos a sesiones ajenas en las pruebas HTTP de autorización (TA-011).	3.6

1.5 Alcance y limitaciones declaradas

Incluye:

- Cinco especialidades: medicina interna, pediatría, ginecología y obstetricia, cirugía general y emergencias.
- Interacción por voz en el navegador, tablilla que se llena en tiempo real y reporte final con puntaje y errores clasificados.
- Pipeline de preparación de casos con validador automático.
- Un médico asistente virtual que presenta el caso, un paciente y agentes de laboratorio, radiología y especialista.
- Panel Doctor para importar casos, crear grupos, asignar actividades y consultar entregas.

Excluye:

- Datos de pacientes reales y cualquier uso para diagnóstico o atención real.
- Desarrollo propio de reconocimiento de voz o de sincronización labial.
- Aplicación móvil nativa.
- Certificación académica de los resultados del estudiante.

Restricciones:

- Hardware: sin servidores propios; las funciones serverless de Vercel tienen límite de tiempo, y un turno con el modelo tardó de 10 a 13 s en los experimentos.
- Datos: solo datasets públicos y casos sintéticos propios. MedQA está en inglés y no trae tratamiento ni explicación de los diferenciales, por lo que necesita conversión y curación.
- Tiempo: el de un taller universitario, con una velocidad estimada de 1 a 1,5 SP por persona por semana.
- Acceso al dominio: el equipo no incluye a un médico; la curación clínica de los casos se respalda en fuentes clínicas citadas (MedlinePlus, CDC y guías), sin revisión médica humana.

1.6 Contribución técnica principal *(research contribution)*

- La contribución es una combinación de técnicas aplicada a la educación médica en español, bajo los siguientes puntos:
  - Casos canónicos con revelación controlada: cada dato tiene un `evidence_id` estable y una `reveal_policy` que dice qué agente lo revela y ante qué acción del estudiante.

- Aislamiento de conocimiento por rol: cada agente recibe solo los datos de su rol, y el diagnóstico nunca entra al contexto del modelo. En los experimentos esto redujo las fugas de 3 de 4 a 0 de 4 y el contexto de 10 481 a 5 098 tokens.
- Evaluación determinística del proceso: el reporte clasifica cada error como negligencia, anamnesis incompleta o razonamiento clínico.
- Conversión de MedQA con reglas y validador: los casos que pasaron todas las verificaciones subieron de 0 % (sin reglas) a 98 % (con reglas y validador).

SECCIÓN 2 — REVISIÓN DE LITERATURA TÉCNICA

(Equivale a Related Work / Background)

2.1 Marco conceptual técnico

Concepto	Definición	Uso en el proyecto
Modelo de lenguaje (LLM)	Red neuronal que genera texto a partir de una instrucción y un contexto. No es determinista: con temperatura 0 su exactitud varió hasta 15 % entre corridas idénticas (Atil et al., 2024).	Solo redacta las respuestas de los personajes. Nunca decide qué se revela ni evalúa.
Paciente virtual o simulado	Programa que representa a un paciente para que un estudiante practique la entrevista clínica.	Paciente y médico asistente con avatar y voz.
Agente y sistema multiagente	Un agente usa el LLM para cumplir un rol; varios agentes se reparten la conversación.	Agentes de paciente, laboratorio, radiología y especialista.
Orquestación con grafo de estados (StateGraph)	El flujo se define como un grafo de pasos que comparten un estado; cada paso decide el siguiente.	Dirige cada turno al agente correcto con una sola llamada al modelo.
Generación aumentada con recuperación (RAG) y recuperación selectiva	Se recupera información de una fuente y se entrega al modelo como contexto, en vez de confiar en lo que recuerda. La variante selectiva entrega solo el fragmento permitido.	Cada agente recibe solo los datos de su rol y su momento.
Principio de mínimo privilegio	Cada componente recibe solo el acceso que necesita.	El diagnóstico nunca entra al contexto del modelo.
Motor de reglas y máquina de estados	Código que aplica reglas fijas: la misma entrada siempre da la misma salida.	Alergias, contraindicaciones, revelación de datos y evaluación.
JSON Schema	Estándar para describir y validar la estructura de documentos JSON.	Esquema de los casos (EN-001).
ETL con Quality Gates	Extraer, transformar y cargar datos con controles que aceptan o rechazan cada registro.	Preparación de los casos antes de usarlos.
Web Speech API	API del navegador para reconocimiento y síntesis de voz.	Voz del estudiante y de los personajes.

2.2 Estado del arte de soluciones similares

Tabla obligatoria con mínimo 15 trabajos de los últimos 5 años:

Ref	Año	Tipo de solución	Técnica/Tecnología	Dataset/Contexto	Métrica principal	Limitación reportada

[1]	2025	Paciente virtual con agentes y tutor.	Gemini 1.5 Flash, Whisper en el navegador, Unity WebGL.	ECOE neurológico; 18 participantes (8 médicos).	Paciente 4,38/5 y tutor 4,33/5; 2,07 s por respuesta de 50 palabras.	Muestra pequeña y un solo escenario; depende de modelos en la nube.
[2]	2025	Benchmark de agentes clínicos.	Entorno de historia clínica compatible con FHIR.	300 tareas, 100 pacientes.	Mejor modelo 69,67 % de éxito (Claude 3.5 Sonnet v2).	Gran variación entre categorías de tareas.
[3]	2025	Evaluación de LLM en examen médico.	22 LLM generales y afinados.	210 preguntas del MIR 2024 y 2025v en España.	Los modelos afinados superan a los generales.	Caída de rendimiento con preguntas modificadas para evitar memorización.
[4]	2025	Paciente estandarizado simulado.	Coevolución de agentes paciente y médico.	200 casos, unas 10 h de simulación.	Más de 10 % de mejora en alineación con los requisitos.	No declarada en el resumen del artículo.
[5]	2025	Simulador multiagente de consulta.	Médico LLM con paciente, examinador y jefe simulados.	506 historias clínicas en chino.	Diagnóstico de GPT-4: 29,36 en varios turnos frente a 58,89 en un paso.	Datos solo en chino; la simulación no replica toda la interacción real.
[6]	2024	Chatbot como paciente simulado.	GPT-3.5 con guion de la enfermedad.	28 estudiantes, 826 pares de pregunta-respuesta, en alemán.	97,9 % de respuestas plausibles; usabilidad 77/100.	Fuera del guion, 56,4 % de las respuestas usaron información inventada; un solo caso; solo texto.
[7]	2024	IA de diálogo diagnóstico.	LLM entrenado con autojuego en entorno simulado.	149 escenarios tipo ECOE frente a 20 médicos.	Superior en 28 de 32 ejes según especialistas.	El chat de texto no representa la práctica clínica habitual.
[8]	2024	Benchmark de agentes en entornos clínicos.	Decisión secuencial multimodal con herramientas.	MedQA convertida; 9 especialidades, 7 idiomas.	En formato secuencial, la exactitud cae a menos de un décimo de la original.	Los benchmarks estáticos no capturan la toma de decisiones secuencial.
[9]	2024	Paciente simulado con evaluación.	LLM con grafo del caso y calificación automática.	8 casos en chino, 80 diálogos.	Correlación con expertos 0,81 (Spearman) y 0,85 (Pearson).	Cuando falta información en el guion, el modelo inventa detalles.
[10]	2024	Paciente simulado para salud mental.	LLM con modelos cognitivos de terapia cognitivo-conductual.	13 aprendices y 20 expertos.	Los expertos lo juzgaron más cercano a pacientes reales que GPT-4.	No declarada en el resumen del artículo.
[11]	2024	Benchmark multilingüe con explicaciones.	RAG y ajuste fino sobre LLM abiertos.	CasiMedicos: 622 casos del MIR (España) en 4 idiomas.	Hasta 89,6 % en inglés con explicaciones de referencia (Mistral 7B).	Rendimiento menor en español, francés e italiano; el RAG aporta poco.
[12]–[15]	**[PENDIENTE: agregar 4 trabajos más para cumplir el mínimo de 15]**					

Referencias identificadas (verificar autores y títulos con el gestor bibliográfico del equipo): [1] Voigt et al., 2025 · [2] Jiang et al., 2025 (MedAgentBench) · [3] **[autores por completar]** · [4] Du et al., 2025 (EvoPatient) · [5] **[autores por completar]** · [6] Holderried et al., 2024 · [7] Tu et al., 2024 (AMIE) · [8] Schmidgall et al., 2024 (AgentClinic) · [9] Li et al., 2024 · [10] Wang et al., 2024 (Patient-Ψ) · [11] Alonso et al., 2024 (MedExpQA).

2.3 Análisis comparativo de gaps

Trabajo	Entrena a un estudiante	Voz	Revelación de datos controlada por reglas	Diagnóstico fuera del contexto del modelo	Evaluación determinística del proceso
[1] Voigt et al.	Sí	Sí	No reportado	No reportado	No: tutor con LLM
[4] Du et al.	Sí	No reportado	No reportado	No reportado	No reportado
[5]	No: evalúa modelos	No	Parcial: examinador simulado	No reportado	No
[6] Holderried et al.	Sí	No: texto	Parcial: guion, pero inventa fuera de él	No reportado	No evalúa
[7] Tu et al.	No: la IA es el médico	No: texto	No aplica	No aplica	No: evaluación humana
[8] Schmidgall et al.	No: evalúa agentes	No reportado	Parcial: decisión secuencial	No reportado	No reportado
[9] Li et al.	Sí	No reportado	Parcial: grafo del caso, pero inventa	No reportado	No: calificación con LLM
[10] Wang et al.	Sí	No reportado	No reportado	No reportado	No reportado
Este proyecto	Sí	Sí	Sí: `reveal_policy` por dato	Sí: medido, 0 de 4 fugas	Sí: motor de reglas

- Posicionamiento explícito: Ningún trabajo revisado combina las seis características. Los más cercanos ([1], [6], [9]) entrenan a estudiantes, pero evalúan con un LLM o no evalúan, y no impiden por diseño que el modelo invente o vea el diagnóstico. Este proyecto ocupa ese espacio: conversación natural por voz en español, con el contenido clínico y la evaluación bajo control determinístico. Su debilidad frente a la literatura es la escala: los experimentos usaron un caso y una repetición.

2.4 Justificación de la elección tecnológica

- El stack cambió respecto del planteamiento inicial (React, ChromaDB y LangChain, Claude o Gemini) a medida que los experimentos y el prototipo dieron evidencia. Cada elección se comparó con sus alternativas:

Decisión	Elegida	Alternativas descartadas	Evidencia
Orquestación de agentes	StateGraph	Supervisor central, Handoffs, Supervisor + Handoffs.	Cobertura de evidencia 78,57 %, routing 100 %, 1 llamada por turno y 12,8 s de latencia media, frente a 2 o 3 llamadas y 25 a 38 s (experimento SP-001).
Acceso a los datos	Recuperación selectiva por rol	Contexto compartido, contexto mínimo, híbrido con guardrails.	0 fugas en 4 ataques, 100 % de rechazos correctos y 5 098 tokens; el contexto compartido filtró el diagnóstico en 3 de 4 ataques y usó 10 481 tokens (experimento SP-003).
Modelo de lenguaje	openai/gpt-oss-20b en GroqCloud, temperatura 0 (configurable)	Claude y Gemini (planteamiento inicial).	Usado en los dos experimentos dentro del plan gratuito, con límite de 200 000 tokens diarios. Pendiente: comparar modelos con los mismos casos.

Conversión de casos	LLM con reglas y validador (`ia_validador`)	Sin reglas, redacción manual.	Casos válidos: 0 de 60 sin reglas, 54 de 60 con reglas y 59 de 60 con reglas y validador (experimento SP-002).
Fuente de casos	MedQA (principal) y casos sintéticos del equipo en familias de diagnóstico diferencial (complemento)	MedMCQA; Synthea (pospuesto: no trae la conversación clínica que exige el simulador).	48 % de preguntas aptas en MedQA frente a 5 % en MedMCQA; 3 083 preguntas útiles (SP-002). El pipeline final leyó 28 492 preguntas y dejó 4 790 candidatas. En evaluación: CasiMedicos, en español y con explicaciones de los diferenciales.
Interfaz	HTML, JavaScript y Tailwind CSS 4	React	Desarrollo rápido; accesibilidad en Lighthouse entre 96 y 100 en las 5 páginas, en móvil y escritorio (TA-024, reporte 1).
Servidor	FastAPI en funciones serverless de Vercel	Servidor propio	Despliegue gratuito; hay que vigilar el límite de tiempo por función.
Voz	Reconocimiento: Web Speech API. Síntesis: edge-tts con voz por personaje y respaldo en la voz del navegador	Whisper como API externa; solo la voz del navegador.	Reconocimiento gratuito y sin servidor; solo funciona en Chrome y Edge. Síntesis: primer fragmento con mediana de 0,95 a 0,99 s y pico de 3,69 s (SP-006). Azure AI Speech F0 elegido como reemplazo, pendiente de activación.
Avatar	Video en bucle de LivePortrait alternado con la imagen fija, sincronizado por el volumen del audio	Avatar 3D; sincronización labial en tiempo real.	16 de 16 comprobaciones en paciente hombre y mujer; respaldos sin video 7 de 7 y solo texto 8 de 8 (SP-006, TA-020).

SECCIÓN 3 — DISEÑO DE LA SOLUCIÓN TECNOLÓGICA

3.1 Visión general de la arquitectura

- Diagrama de arquitectura de alto nivel (obligatorio, exportable como figura vectorial)

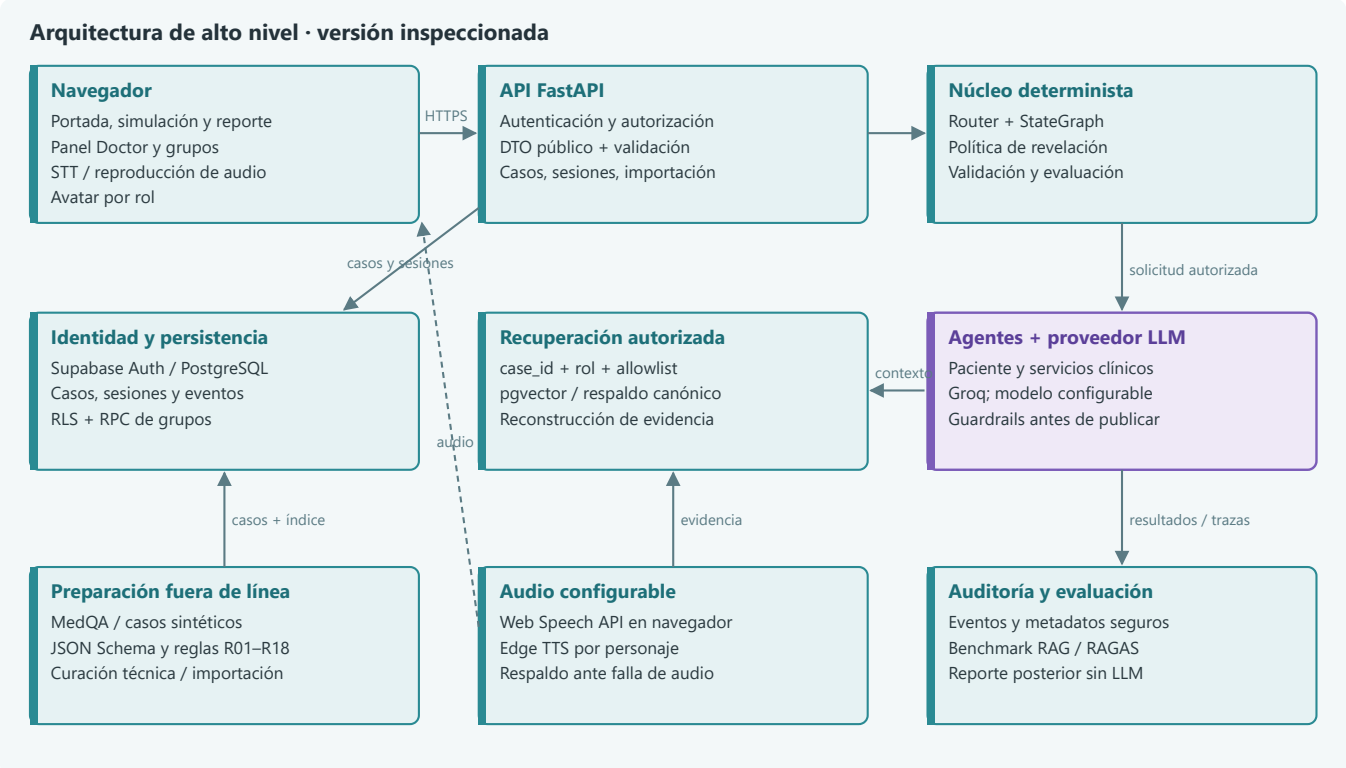


Figura 1. Arquitectura de alto nivel.

- Descripción de cada componente y su responsabilidad

Componente	Responsabilidad y límite
Frontend Web	Selección, chat, voz, avatar, hipótesis, ficha, reporte, catálogo y grupos; no recibe `protected` durante la consulta.
API FastAPI	Validación de peticiones, autorización, adaptación del estado a vistas públicas y manejo de errores.
Router y StateGraph	Selección de intención y agente, y avance de fase; un agente destinatario por intervención.
Política de revelación	Construye `allowed_evidence_ids` según caso, dueño, modo y solicitud.
Repositorios y RAG	Acceso canónico o remoto; búsqueda limitada al caso, al rol y a la allowlist.
Agentes y gateway	Redacción por personaje, inferencia remota configurable y recuperación ante errores de proveedor.
Motor clínico	Reglas de seguridad, evaluación de evidencia y construcción del reporte al finalizar.
Supabase	Auth, casos, complementos, sesiones, eventos, grupos, asignaciones y registro de diagnósticos.
Preparación de casos	Conversión, normalización, esquema, reglas cruzadas, revisión técnica e indexación.
Evidencia y benchmark	Pruebas, trazas, métricas por identificadores y evaluación RAGAS.

3.2 Especificación de requerimientos técnicos

3.2.1 Requerimientos funcionales

ID	Requerimiento	Prioridad	Vinculado a objetivo
RF01	Autenticar usuarios y verificar el rol docente desde el servidor.	Crítica	OE5
RF02	Entregar únicamente la vista pública de un caso seleccionado.	Crítica	OE1, OE2
RF03	Dirigir cada pregunta al paciente o servicio correspondiente.	Alta	OE2
RF04	Revelar datos solo si su política y el caso activo lo autorizan.	Crítica	OE2, OE5
RF05	Evitar el turno intermedio que repite la pregunta del estudiante.	Alta	OE2, OE4
RF06	Solicitar exploración, laboratorio, imágenes e interconsulta; aclarar peticiones genéricas.	Alta	OE2, OE4
RF07	Completar datos ausentes únicamente dentro de contratos habilitados, con procedencia y sin sobrescribir el dato canónico.	Alta	OE1, OE2
RF08	Registrar hipótesis y confirmar el diagnóstico antes de cerrar.	Alta	OE3
RF09	Generar un reporte poscierre con referencia, proceso, evidencia y omisiones.	Alta	OE3
RF10	Importar JSON/Excel, prevalidar y publicar casos con sus evidencias.	Alta	OE1, OE4
RF11	Consultar y sincronizar el catálogo general de evidencias.	Alta	OE1, OE4
RF12	Crear grupos con nombre único por doctor y añadir cuentas registradas.	Alta	OE4, OE5
RF13	Asignar actividades y consultar entregas, tiempos y puntajes del proceso.	Alta	OE3, OE4
RF14	Conservar sesiones y eventos entre instancias de servidor.	Crítica	OE5

RF15	Retomar una sesión interrumpida desde un listado de sesiones abiertas. *Estado: implementado solo para actividades de grupo; caso general en desarrollo (HU-009).*	Media	OE4, OE5
RF16	Calificar automáticamente la propuesta por referencia/sinónimos.	Media	OE3
RF17	Medir recuperación, latencia y errores con artefactos reproducibles.	Alta	OE2, OE5

3.2.2 Requerimientos no funcionales

- Rendimiento, escalabilidad, seguridad, usabilidad, portabilidad

Dimensión	Requisito y forma de verificar
Rendimiento	Medir p50, p90, p99, media, desviación, throughput y errores por endpoint y concurrencia; los resultados se reportan en 5.4 (TA-024).
Escalabilidad	Usar persistencia externa para varias instancias; separar el estado de sesión del proceso y verificar la concurrencia por actividad.
Seguridad	Propietario/capacidad en todas las operaciones; rol verificado y salida restringida al caso; 401/404 sin contenido ajeno.
Fiabilidad	Reintentos controlados, cierre idempotente y manejo explícito de servicios caídos; no sustituir un caso remoto con otro local.
Usabilidad	Emisor visible, ausencia de eco del doctor, avisos comprensibles y reporte poscierre; piloto/SUS aún pendientes.
Accesibilidad	Navegación, etiquetas y estados perceptibles; opción «reducir movimiento» sin video; los resultados históricos de navegador no sustituyen una auditoría completa actual.
Portabilidad	Python ≥ 3.12 y Node ≥ 18; configuración por variables de entorno; navegador compatible con voz.
Trazabilidad	IDs estables, huellas de contenido, eventos y artefactos de evaluación; conservar la procedencia de cada resultado.

3.3 Modelado del sistema *(según tipo de solución)*

Para **soluciones de software (web/móvil)**:

- Diagrama de casos de uso

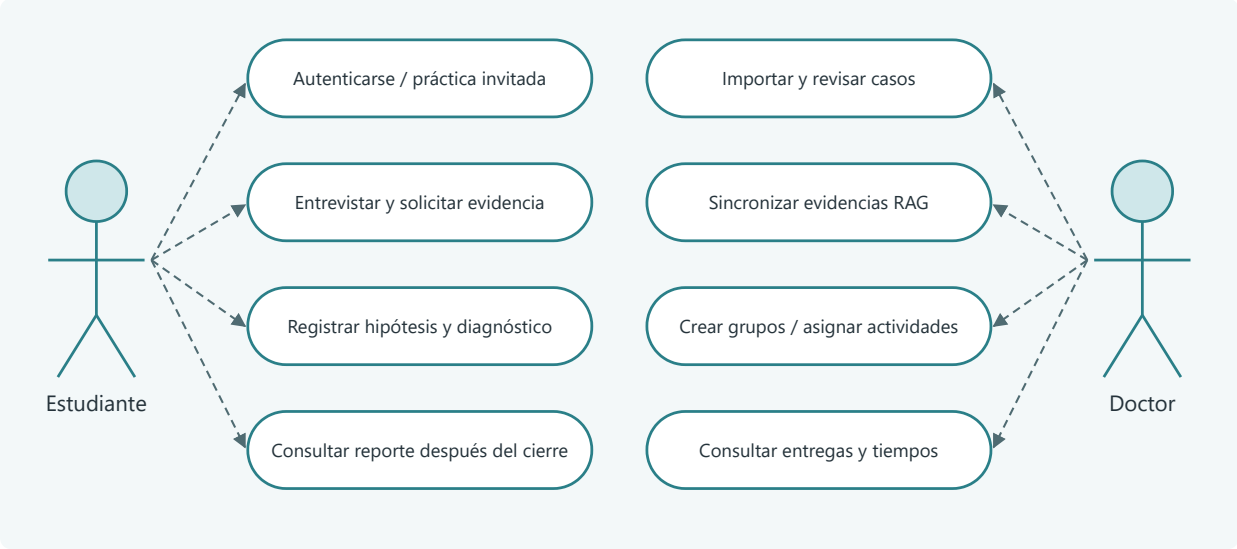


Figura 2. Diagrama de casos de uso.

Para **modelos IA / ML / DL**:

- Pipeline completo de datos → preprocesamiento → modelo → salida

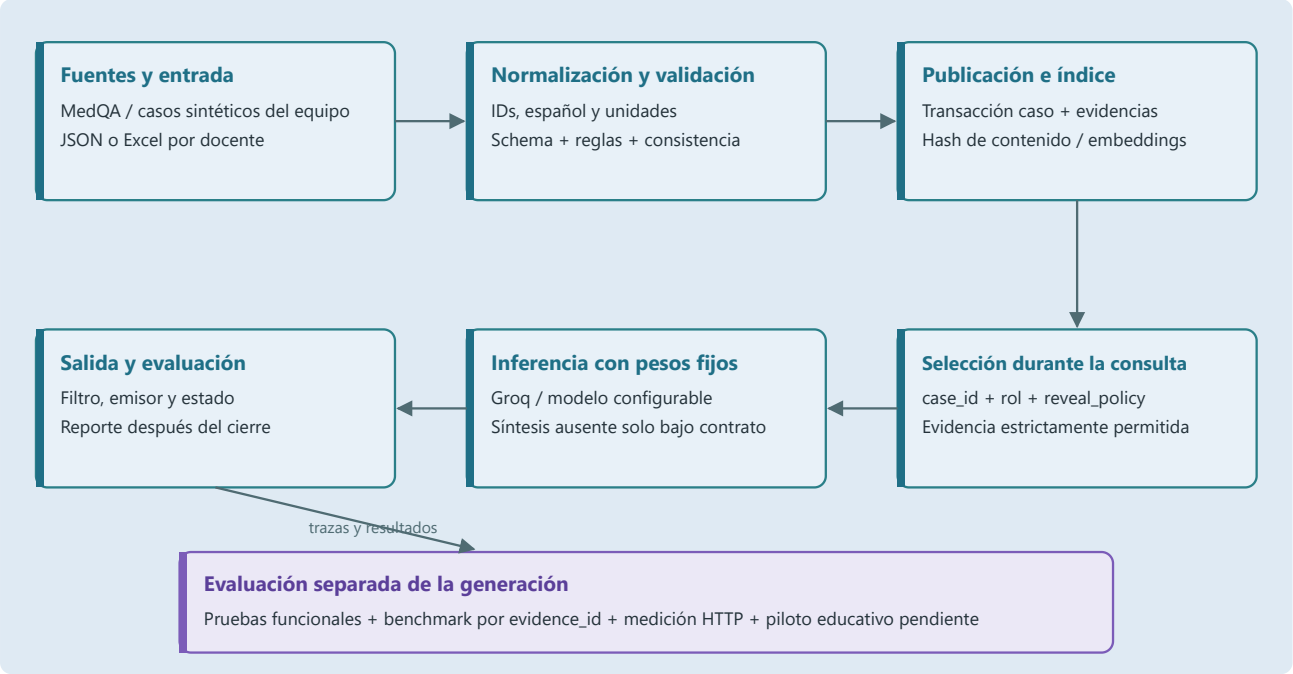


Figura 3. Pipeline de datos, inferencia y evaluación.

- Función objetivo / función de pérdida con notación matemática

A. Entropía cruzada autorregresiva · formulación teórica

$$\mathcal{L}_{\mathrm{AR}}\left(\theta;\mathcal{D}\right) = -\frac{1}{N}\sum_{n=1}^{N}\frac{1}{T_n}\sum_{t=1}^{T_n}\log P_\theta\left(y_{n,t} \mid y_{n,<t}, q_n, C_n\right)$$

N: número de ejemplos; T_n: tokens de la respuesta de referencia del ejemplo n. q_n: pregunta; C_n: contexto; $y_{n,t}$: token esperado; θ: pesos del modelo.

En este proyecto: θ = θ_0 (pesos fijos). No se minimiza esta pérdida ni se hace fine-tuning.

B. Función implementada · puntaje del proceso de consulta

$$S = \mathrm{round}\left(100\frac{\sum_{i\in Q} w_i \mathbf{1}\left[E_i \subseteq O\right]}{\sum_{i\in Q} w_i}, 2\right)$$

Q: preguntas evaluables de la rúbrica; w_i: peso; E_i: evidencias requeridas; O: evidencias observadas. La selección de Q considera la evidencia entregada al inicio. S mide el proceso, no el acierto diagnóstico.

Como formulación teórica de un modelo autorregresivo condicionado, la entropía cruzada mide el negativo del logaritmo de la probabilidad de los tokens de referencia. D representa un conjunto de N ejemplos, T_n su longitud, q_n la pregunta, C_n el contexto y θ los pesos. Se incluye para expresar matemáticamente el componente generativo; no describe una función entrenada por este proyecto ni pretende resumir todas las etapas de entrenamiento del proveedor. Aquí se usa θ fijo y se valida la respuesta respecto del contexto autorizado. La función efectivamente implementada en la aplicación es el puntaje ponderado de evidencia del proceso, mostrado en la segunda parte de la figura, no una pérdida de entrenamiento ni una nota automática de acierto diagnóstico.

No existe una función de pérdida optimizada durante la ejecución del prototipo. Sea θ el conjunto de pesos del proveedor, que permanece fijo; la respuesta se obtiene de $P_\theta(y \mid q, C_{aut})$, donde C_{aut} contiene solo el contexto autorizado. La función determinista $V(y, C_{aut})$ decide si se permite publicar la salida o si se utiliza una respuesta de respaldo que no contiene términos protegidos.

3.4 Stack tecnológico justificado

Capa	Tecnología elegida	Versión	Justificación técnica	Alternativa descartada
Frontend	HTML, JavaScript; Tailwind CSS	HTML5; JavaScript del navegador; Tailwind 4.3.3	Interfaz con estilos compilados y controles nativos; integra las vistas existentes sin añadir un framework SPA.	React: no se adopta en la versión actual para evitar una reescritura de las vistas.
Backend/API	Python, FastAPI y Uvicorn	Python ≥ 3.12; FastAPI 0.141.1; Uvicorn 0.54.0	Validación de peticiones y ejecución ASGI; conserva el motor clínico en Python.	Flask: implicaría adaptar los contratos y la integración ASGI existentes.
Orquestación	LangGraph	1.2.14	Grafo explícito por rol y transiciones controladas desde código.	Encadenamiento libre de prompts: dificulta limitar fases y responsabilidades.
Modelo de lenguaje	Groq API; openai/gpt-oss-20b configurable	Identificador de modelo gpt-oss-20b; API compatible con Chat Completions, sin versión fijada en el cliente	Inferencia externa mediante un adaptador; no requiere alojar ni entrenar pesos en el equipo.	Alojamiento local de los pesos: añade requisitos de memoria y operación no contemplados en el despliegue.
Autenticación	Supabase Auth y supabase-js	supabase-js 2.117.2; servicio gestionado sin versión exportada	Identidad centralizada y tokens que el backend verifica antes de autorizar acciones.	Autenticación propia: añade gestión de contraseñas y recuperación de cuentas.
Persistencia	PostgreSQL en Supabase	Versión del servidor no verificada	JSON, claves, restricciones, RPC y políticas de acceso para casos, sesiones y grupos.	Archivos JSON como persistencia de producción: dificultan concurrencia y estado entre instancias.
Recuperación	pgvector y adaptadores RAG del proyecto	Versión remota de pgvector no verificada; adaptadores de la copia inspeccionada	Búsqueda limitada por caso, rol y allowlist; respaldo canónico disponible.	Contexto global sin filtros: expone información ajena al permiso del turno.
Voz	Web Speech API y Edge TTS	API del navegador; edge-tts 7.3.1	Reconocimiento disponible en el navegador y voces por personaje con respaldo.	Solo síntesis del navegador: menor control de la voz elegida entre equipos.
Avatar	Video de LivePortrait + imagen fija	Videos H.264 pregenerados (334 a 408 KB)	Se reproduce en todos los navegadores probados sin cómputo en el cliente; sincronía por volumen.	Sincronización labial en tiempo real: requiere GPU o un servicio de pago.
Validación e importación	jsonschema, openpyxl y defusedxml	4.23.0; ≥3.1.5,<4; ≥0.7.1,<1	Contratos JSON verificables y lectura de Excel con límites de estructura.	Importación sin esquema: permite documentos incompletos o incompatibles con el motor.

Evaluación y pruebas	Métricas por IDs, RAGAS y pytest	Métricas de la copia inspeccionada; RAGAS 0.4.3 opcional; pytest 8.3.4; Playwright 1.63.0	Separa controles de recuperación, generación y regresión; RAGAS puede desactivarse.	Una sola nota de un LLM: mezcla criterios y reduce trazabilidad.
Despliegue	Vercel	Servicio gestionado; configuración en vercel.json, sin versión de plataforma exportada	Publica frontend y API con la configuración del repositorio.	Servidor administrado por el equipo: exige operación y mantenimiento adicionales.

3.5 Decisiones de diseño críticas

- Mínimo 3 decisiones arquitectónicas documentadas como ADR (Architecture Decision Record) simplificado:
  - ADR-01. Separar redacción y decisiones clínicas
    - **Decisión tomada:** La intención, la revelación, el estado y la evaluación se resuelven en código; el modelo redacta con un contexto limitado.
    - **Contexto que la motivó:** Un modelo puede producir texto plausible sin respetar permisos ni la rúbrica.
    - **Alternativas evaluadas:** Solicitar al mismo LLM que decida el acceso y la nota; utilizar un guion fijo para toda la consulta.
    - **Consecuencias asumidas:** Aumenta la posibilidad de probar invariantes y repetir una evaluación; exige reglas y contratos explícitos y no garantiza por sí sola validez médica.
  - ADR-02. Recuperar por evidencia autorizada, no por contexto global
    - **Decisión tomada:** Autorizar primero y restringir por case_id, dueño y evidence_id; el orquestador vuelve a intersectar las referencias y reconstruye el contenido canónico.
    - **Contexto que la motivó:** Un índice puede devolver contenido irrelevante, antiguo o de otro caso.
    - **Alternativas evaluadas:** Entregar el documento completo, limitar el prompt sin control de datos o confiar en el ranking remoto.
    - **Consecuencias asumidas:** Reduce la superficie de exposición y permite pruebas adversariales; si el conjunto permitido es incompleto o el k insuficiente, la recuperación puede omitir hechos pertinentes.
  - ADR-03. Mantener casos, complementos y estado como entidades distintas
    - **Decisión tomada:** Conservar el caso en `casos_curados`, los complementos de historia en `caso_datos_adicionales` y los resultados sintéticos de exámenes en el snapshot de sesión.
    - **Contexto que la motivó:** Un dato generado durante una entrevista no debe modificar el documento fuente como si siempre hubiera formado parte de él.
    - **Alternativas evaluadas:** Reescribir el JSON canónico o guardar todo en una columna de conversación.
    - **Consecuencias asumidas:** Se conserva la procedencia y se evita regeneración contradictoria dentro del alcance implementado; hace falta una política humana de revisión y de reutilización más amplia.
  - ADR-04. Sesiones persistentes y autorizadas por propietario
    - **Decisión tomada:** Guardar sesiones y eventos en Supabase y autorizar cada operación con `session_id` por propietario: el token verificado del alumno o una credencial de capacidad de 256 bits para el invitado, almacenada solo como hash.

    - **Contexto que la motivó:** En Vercel cada petición puede atenderla otra instancia, y la sesión en memoria se perdía (HTTP 404). Además, la auditoría TA-011 mostró que cualquiera que conociera el UUID de una consulta podía leerla, escribir en ella o cerrarla.
    - **Alternativas evaluadas:** Sesiones en memoria; el UUID como único secreto; autorización solo con RLS desde el navegador.
    - **Consecuencias asumidas:** La sesión sobrevive al cambio de instancia y una sesión ajena devuelve 404 sin datos; cada operación depende de Supabase (503 si no responde) y el invitado que pierde su credencial no puede recuperar la consulta.
- ADR-05. Tratar la síntesis como una capacidad acotada y trazable
    - **Decisión tomada:** Habilitar componentes de historia y exámenes con contratos, validación, marca sintética y prioridad del dato canónico.
    - **Contexto que la motivó:** La ficha no contiene todas las respuestas posibles, pero improvisar cualquier resultado puede contradecir el caso o inducir el diagnóstico.
    - **Alternativas evaluadas:** Negar sistemáticamente cualquier dato ausente o generar contenido sin control.
    - **Consecuencias asumidas:** Mejora la continuidad de las consultas soportadas; un contrato automático no convierte el dato en una observación real ni en un resultado validado por un médico.
- ADR-06. Diálogo directo y voz con degradación funcional
    - **Decisión tomada:** Enviar la pregunta original al destinatario y mantener al doctor para presentación, hallazgos y aclaraciones. El TTS neuronal se configura por personaje y utiliza respaldo del navegador.
    - **Contexto que la motivó:** Repetir cada pregunta con la voz del doctor añade un turno sin nueva evidencia y alarga la interacción. Los servicios de audio también pueden fallar.
    - **Alternativas evaluadas:** Reformulación audible obligatoria o dependencia exclusiva de un proveedor de audio.
    - **Consecuencias asumidas:** El emisor de cada respuesta queda claro; es necesario probar la reproducción de eventos antiguos y los fallos de voz. La disponibilidad del STT sigue dependiendo del navegador y sus permisos.

3.6 Modelo de seguridad y privacidad

- Mecanismos de autenticación, autorización, cifrado

Autenticación. Supabase Auth gestiona las cuentas, las contraseñas con hash y la recuperación. El backend no confía en el navegador: verifica cada token `Bearer` contra el servicio de Supabase antes de atender la petición (401 si el token no es válido o venció; 503 si Supabase no responde). El rol se toma solo de `app_metadata.rol`, que asigna la administración; cualquier otro valor se trata como alumno. Las consultas de invitados reciben una credencial de capacidad de 256 bits (`secrets.token_urlsafe(32)`), de la que el servidor guarda solo el hash SHA-256 y que compara en tiempo constante.

Autorización.

Recurso	Control
Sesión del simulador (mensajes, eventos, cierre, reporte)	Solo la abre su propietario. Sin credencial: 401. Sesión ajena o inexistente: el mismo 404, para no revelar cuáles existen. Las denegaciones se registran con la credencial enmascarada.
Panel Doctor, importación y grupos	Requiere rol profesor o administrativo verificado en el servidor (403 en otro caso).

Datos protegidos del caso	Diagnóstico, diferenciales, tratamiento y rúbrica solo los leen el motor de reglas y la evaluación; el estudiante recibe la vista pública.
Contexto del modelo	Mínimo privilegio: el agente recibe solo `allowed_evidence_ids` del caso activo y de su rol. Una prueba automática impide que la capa de IA importe los módulos de casos o del motor.
Salida del modelo	Un guardrail bloquea términos protegidos (diagnóstico y sinónimos) y los reemplaza por una respuesta de respaldo sin revelar qué término se bloqueó.
Base de datos	RLS activado en todas las tablas; acceso directo revocado a `anon` y `authenticated` en sesiones, casos, importaciones, grupos y diagnósticos; funciones `security definer` con `search_path` vacío; solo el backend accede con `service_role`.
Importación de archivos	Máximo 4 MB, 100 casos y 5 000 filas; Excel leído con defusedxml; validación de esquema y reglas antes de publicar.

Cifrado. En tránsito, HTTPS/TLS en Vercel, Supabase y Groq. En reposo, el cifrado del proveedor administrado (Supabase); no se implementa cifrado propio por columna. Las claves viven solo en `.env` o en las variables de Vercel: la clave secreta de Supabase se usa solo en el backend y el navegador recibe únicamente la URL y la clave pública.

- Consideraciones de privacidad de datos (especialmente si hay datos personales o sensibles)
  - No se usan datos de pacientes reales: los casos vienen de preguntas de examen (MedQA) o son sintéticos.
  - De los alumnos con cuenta se guardan nombres, correo, puntaje, diagnóstico, hipótesis, duración y la conversación. El modo invitado no guarda nada que identifique al alumno.
  - Aviso de privacidad según la Ley N.º 29733 (Perú): crear una cuenta exige aceptarlo y se registran su versión y fecha. El aviso informa los encargados del tratamiento (Supabase, Vercel y Groq), el plazo de conservación y los derechos ARCO.
  - Minimización hacia el proveedor del modelo: Groq recibe solo el texto de la pregunta y la evidencia autorizada, sin nombre ni correo. Las trazas guardan metadatos saneados y LangSmith está desactivado por defecto.

SECCIÓN 4 — DESARROLLO E IMPLEMENTACIÓN

(Equivale a Implementation)

4.1 Metodología de desarrollo aplicada

- Justificación de la metodología (Scrum, Kanban, DSR, prototipado, etc.)

Se usó **Scrum** con un tablero de backlog organizado por fases del proyecto (1 Definición del problema, 2 y 3 Preparación de los datos, 4 Modelamiento/Aprendizaje, 5 Evaluación, 6 Paso a producción) y elementos tipificados: historias de usuario (HU), tareas técnicas (TA), habilitadores (EN) y spikes de investigación (SP). Los requisitos cambiaron con la evidencia —el stack de 2.4 es el resultado de esos cambios—, por lo que cada decisión técnica se validó antes de construir sobre ella. Por eso Scrum se combinó con **spikes experimentales** (SP-001 a SP-006), un enfoque cercano a *Design Science Research*: construir un artefacto, medirlo y ajustarlo. Cada entrega queda respaldada por una carpeta de evidencias con objetivo, comandos, salida real y los intentos fallidos conservados.

- Sprints o iteraciones realizadas con entregables por fase

Iteración	Período	Entregables principales
Sprint 0 · Investigación	Hasta 06/10/2026	SP-001 arquitecturas de agentes, SP-002 conversión de datasets, SP-003 aislamiento de conocimiento, SP-004 anclaje a evidencia, SP-005 estrategia RAG.

Sprint 1 · Datos y núcleo	06/10 – 22/10/2026	EN-001 esquema JSON, TA-001 pipeline ETL y curación, TA-002 validación de casos, TA-003 prototipo web, EN-002 repositorio y API de casos, EN-003 orquestador con StateGraph, TA-005 primer flujo frontend–backend–IA, TA-007 motor de sesión y routing, TA-008 grounding por agente, TA-004 eventos y estado de sesión, HU-001, HU-002, HU-004 historial, TA-010 ampliación a 43 casos.
Sprint 2 · Seguridad, docencia y experiencia	23/10 – 06/11/2026	TA-011 autorización de sesiones, TA-012 validación adversarial, TA-014 validador de consistencia, HU-005 exámenes y especialistas, EN-005 exámenes sintéticos, TA-006 RAG con pgvector, HU-003/EN-004/HU-004 panel docente e importación, HU-011 grupos y desempeño, TA-021 calificación diagnóstica, TA-023 métricas RAG y RAGAS, HU-006 diagnóstico final, HU-007 reporte final, SP-006, TA-019 y TA-020 voz y avatar.
En curso	Desde 06/11/2026	TA-024 latencia y observabilidad, HU-009 retomar sesiones, HU-008 retroalimentación formativa, TA-022 calidad del razonamiento, TA-013 plausibilidad de casos, TA-025 integración final.

[PENDIENTE: confirmar las fechas oficiales de cada sprint.]

4.2 Descripción técnica de módulos implementados

Por cada módulo principal:

M1 · Router determinístico (`backend/server/orchestration/router.py`)

- Función técnica: clasificar cada mensaje en una intención y un agente destinatario sin usar el modelo de lenguaje.
- Decisión no trivial: el orden de las reglas importa (cierre, diagnóstico y tratamiento antes que exámenes) y una pregunta con «?» nunca es una propuesta de tratamiento («¿Qué le recetaron?» va al paciente). Si ninguna regla aplica, va al paciente con confianza 0,75.

```
def route(self, message: str) -> RouteDecision:
    text = _normalize(message)                      # minúsculas, sin tildes ni signos
    def mentions(terms):
        return any(f" {t} " in f" {text.strip()} " for t in terms)
    if mentions(self.FINISH):                       # 1) cierre explícito
        return RouteDecision(Intent.FINISH, AgentRole.MEDICO_ASISTENTE, 1.0, "...")
    if mentions(self.DIAGNOSIS):                    # 2) propuesta de diagnóstico
        return RouteDecision(Intent.PROPOSE_DIAGNOSIS, AgentRole.MEDICO_ASISTENTE, 0.98, "...")
    if "?" not in message and mentions(self.TREATMENT):  # una pregunta no es una propuesta
        return RouteDecision(Intent.PROPOSE_TREATMENT, AgentRole.MEDICO_ASISTENTE, 0.98, "...")
    if mentions(self.LAB):                          # 3) exámenes por servicio
        return RouteDecision(Intent.REQUEST_LAB, AgentRole.LABORATORIO, 0.95, "...")
    if mentions(self.RADIOLOGY):
        return RouteDecision(Intent.REQUEST_RADIOLOGY, AgentRole.RADIOLOGIA, 0.95, "...")
    # ... especialista, examen físico, pedido genérico, asistente ...
    return RouteDecision(Intent.ASK_PATIENT, AgentRole.PACIENTE, 0.75, "por defecto")
```

M2 · Política de revelación (`backend/server/engine/disclosure/policy.py`)

- Función técnica: calcular `allowed_evidence_ids` del turno. Un dato se revela solo si coinciden su dueño (rol), su modo (preguntar, examinar o solicitar examen) y todos sus requisitos temáticos.
- Decisión no trivial: los requisitos se comparan con alias normalizados («desde cuándo», «hace cuánto» → `inicio_y_evolucion`); un pedido genérico («análisis de sangre») no basta para afirmar que un examen no se realizó: se pide aclarar cuál.

```
def authorize(self, *, case, role, intent, student_message) -> DisclosureDecision:
    mode = MODE_BY_INTENT[intent]                  # p. ej. REQUEST_LAB -> "al_solicitar_examen"
    text = _normalize(student_message)
    allowed, matched = [], []
    for item in case.get("on_request", []):
        policy = item.get("reveal_policy") or {}
        if policy.get("owner") != role.value or policy.get("mode") != mode:
            continue                               # otro rol u otro modo: no se revela
        required = [str(v) for v in policy.get("requires", [])]
        if required and not all(self._requirement_matches(r, text) for r in required):
            continue                               # falta algún tema pedido
        allowed.append(str(item["evidence_id"]))
        matched.extend(required)
    return DisclosureDecision(allowed_evidence_ids=allowed,
                              matched_requirements=list(dict.fromkeys(matched)))
```

M3 · Grafo en estrella (`backend/server/orchestration/star_graph.py`)

- Función técnica: ejecutar el turno como un StateGraph de LangGraph con un orquestador central, un nodo por rol y un nodo de cierre; garantiza una sola llamada al modelo por turno.
- Diagrama de flujo interno:

```
pregunta ─► Router ─► DisclosurePolicy ─► RAG (solo IDs permitidos) ─► Agente(rol) ─► LLM Gateway
                                                                                          │
          tablilla / estado / traza ◄── central_finalize ◄── OutputGuard ◄────────────────┘
                                        (si falla: respuesta de respaldo, sin el término)
```

```
def _build(self):
    builder = StateGraph(TurnGraphState)
    builder.add_node("orchestrator", self._orchestrator_node)    # router + revelación
    for role in AgentRole:                                       # un nodo hoja por agente
        builder.add_node(role.value, self._agent_node(role))
    builder.add_node("central_finalize", self._central_finalize) # guardrail + estado
    builder.add_edge(START, "orchestrator")
    builder.add_conditional_edges("orchestrator", ...)           # va al rol elegido
    # cada agente vuelve al centro; nunca hay arista entre agentes
    builder.add_edge("central_finalize", END)
    return builder.compile()
```

M4 · Gateway del modelo con conmutación (`backend/server/ai/llm/gateway.py`)

- Función técnica: repartir las llamadas entre varias claves o proveedores en rotación circular y aislar los fallos.
- Decisión no trivial: una credencial inválida entra en una espera larga y un error transitorio (límite de uso, tiempo agotado) en una espera configurable; la respuesta servida por un respaldo queda marcada como `fallback_used` en la traza.

```
def _generate(self, messages, *, temperature=0.0, json_mode=False):
    errors = []
    ordered = self._ordered_available() or list(self.providers)  # rotación circular
    for index, provider in enumerate(ordered):
        try:
            result = provider.generate(messages, temperature=temperature)
            if index > 0:                                        # se usó un respaldo
                result = LLMResult(**{**result.__dict__, "fallback_used": True})
            self._advance(provider.name)
            return result
        except ProviderError as exc:
            errors.append(str(exc))
            if exc.invalid_credential:                           # clave inválida: espera larga
                self._cooldown_until[provider.name] = time.monotonic() + INVALID_CREDENTIAL_COOLDOWN_SECONDS
                continue
            if exc.retryable:                                    # 429/timeout: espera corta
                self._cooldown_until[provider.name] = time.monotonic() + self.cooldown_seconds
                continue
            raise
    raise LLMUnavailable("No hay proveedor LLM disponible. " + " | ".join(errors))
```

M5 · Motor de evaluación (`backend/server/engine/evaluation/service.py`)

- Función técnica: calcular el puntaje S del proceso (3.3) y clasificar cada omisión.
- Decisión no trivial: la evidencia que entrega el doctor al presentar el caso no se puntúa ni cuenta como omisión, porque no depende del estudiante.

```
initial = set(initial_evidence_ids or [])
questions = [q for q in all_questions                        # Q: excluye lo ya entregado
             if not set(q["evidence_ids"]).issubset(initial)] or all_questions
observed = set(evidence_ids) | initial                       # O: evidencia observada
total_weight = sum(q["weight"] for q in questions)
earned_weight, errors = 0, {t: 0 for t in ERROR_TYPES}
for q in questions:
    if set(q["evidence_ids"]).issubset(observed):            # 1[E_i ⊆ O]
        earned_weight += q["weight"]
    else:                                                    # omisión clasificada por tipo
        errors[q["error_if_missing"]] += 1
score = round(earned_weight * 100 / total_weight, 2)         # S
```

M6 · Autorización de sesiones (`backend/server/api/ai.py`)

- Función técnica: aplicar la política TA-011 a las rutas que reciben `session_id`.
- Decisión no trivial: una sesión ajena devuelve 404 y no 403, para no confirmar que existe.

```
token = _bearer_token(request, authorization)
if x_session_token is not None and not SESSION_TOKEN_FORMAT.fullmatch(x_session_token):
    raise _deny(request, 401, "credencial_malformada", f"capacidad={_mask(x_session_token)}")
if token is None and x_session_token is None:
    raise _deny(request, 401, "sin_credencial", "anonimo")
user_id = _verified_user(request, token) if token else None   # verificado contra Supabase
try:
    access = get_service().access_details(session_id)
except SessionNotFound as exc:
    raise _deny(request, 404, "inexistente", who) from exc
# alumno: owner_id == user_id ; invitado: hmac.compare_digest(hash, sha256(capacidad))
```

M7 · Pipeline de preparación de casos (`backend/case_preparation/`)

- Función técnica: proceso fuera de línea `sources → adapters → transform → quality_gates → curation → indexing`.

- Decisión no trivial: la fuente se fija por commit y por la huella SHA-256 de cada archivo, y la curación usa fechas fijas; regenerar los casos produce exactamente los JSON versionados.

4.3 Gestión de datos

4.3.1 Fuentes de datos

- Origen, volumen, formato, licencia de uso

Fuente	Origen	Volumen	Formato	Licencia	Uso
MedQA (USMLE)	github.com/jind11/MedQA, commit 27b02f6, `data_clean.zip`; bancos US y Taiwan (inglés)	28 492 preguntas leídas → 4 790 candidatas → 10 casos	JSONL	MIT	Casos de referencia
Casos sintéticos del equipo	Módulos de familias del pipeline	33 casos en 10 familias	JSON	CC BY 4.0	Práctica del diagnóstico diferencial
Fuentes clínicas de los complementos	MedlinePlus, CDC y guías citadas en la revisión de cada caso	Una por complemento	URL	Uso público	Datos que descartan diferenciales
Conjunto de evaluación RAG	Generado de forma determinista desde los casos curados	100 registros	JSON	Propia	Benchmark de recuperación y RAGAS
Casos importados por docentes	Panel Doctor	Hasta 100 casos o 4 MB por archivo	JSON / XLSX	Del docente	Ampliación del catálogo

4.3.2 Preprocesamiento

- Pipeline de limpieza, transformación, normalización
  - Descarga verificada: el ZIP se lee sin descomprimir y se comprueba el SHA-256 de cada archivo; los datos originales nunca se modifican.
  - Filtrado: enunciado de al menos 250 caracteres, con un paciente, que pida diagnóstico o tratamiento, sin depender de una imagen (2 642 excluidas) y que no sea de investigación; sin duplicados y con archivo y línea de origen.
  - Clasificación por especialidad con reglas deterministas (edad < 18 → pediatría; embarazo → ginecología y obstetricia; etc.).
  - Transformación: traducción clínica al español sin inferencias; cada dato se reparte en `public` u `on_request` con su `evidence_id`, dueño, modo y requisitos; diagnóstico, diferenciales y enunciado original van a `protected`.
  - Normalización: IDs `^[A-Z]{3,5}-\d{3}$`, unidades UCUM y al menos tres sinónimos del diagnóstico por caso (52 en total).
  - Quality Gates: JSON Schema y reglas cruzadas R01–R18 (por ejemplo, R13 rechaza cifras ausentes en la fuente y R15 exige un dato que descarte cada diferencial). Los casos rechazados se conservan con su motivo.
  - Publicación: importación transaccional a Supabase y sincronización de evidencias en pgvector.
- Estadísticas descriptivas del dataset (media, desviación, distribución, valores faltantes)

Variable (43 casos curados)	Media ± DE	Mín. – Máx.	Total
Datos públicos iniciales por caso	1,05 ± 0,21	1 – 2	45

Datos revelables a petición	10,47 ± 3,40	3 – 14	450
Diagnósticos diferenciales	2,51 ± 0,83	2 – 4	108
Preguntas clave de la rúbrica	5,88 ± 0,79	4 – 7	253

- Distribución por especialidad: emergencias 11, cirugía general 8, ginecología y obstetricia 8, medicina interna 8, pediatría 8.
- Distribución por origen: MedQA 10 (23,3 %) y sintéticos 33 (76,7 %).
- Evidencias revelables por categoría: anamnesis 130, antecedente 89, laboratorio 49, examen físico 43, síntoma 38, medicación 36, signo vital 31, imagen 30, estudio 4.
- Valores faltantes: el esquema obliga a completar todos los campos. Lo que el enunciado no trae no se rellena, salvo complementos acotados con fuente clínica: en la revisión de los casos de MedQA, 10 de 48 diferenciales no tenían un dato que los descartara y se completaron en 6 casos.

4.3.3 Partición de datos (si aplica ML/DL)

- Train / Validation / Test con proporciones y justificación

No aplica en sentido estricto: el proyecto no entrena ni ajusta pesos ($\theta = \theta_0$, sección 3.3), por lo que no hay partición train/validation/test. Los 43 casos forman el catálogo de práctica; el conjunto de evaluación RAG (100 registros) se genera a partir de todos los casos y mide recuperación, no generalización; los fixtures de prueba están separados de los datos para que las pruebas no dependan del catálogo.

- Estrategia de muestreo (random, stratified, temporal, k-fold)

Selección de MedQA estratificada por especialidad (dos casos por especialidad) con revisión manual; la primera selección automática se rechazó y se conserva como intento fallido. Las familias sintéticas se diseñan por presentación clínica común.

4.4 Configuración del entorno de desarrollo y producción

- Especificaciones de hardware usado

Entorno	Especificación
Desarrollo y pruebas	Intel Core i5-10400 a 2,90 GHz (6 núcleos, 12 hilos), 23,8 GB de RAM, Windows 10 Pro; es también el equipo de las mediciones de TA-024. **[PENDIENTE: equipos de los demás integrantes]**
Generación de avatares (una vez)	Google Colab con GPU T4 (LivePortrait)
Producción	Funciones serverless de Vercel; Supabase administrado; inferencia remota en GroqCloud

- Versiones exactas de dependencias (archivo requirements.txt / package.json referenciado)
  - Python: `backend/pyproject.toml` y `backend/uv.lock`; `pyproject.toml` de la raíz para Vercel. fastapi 0.141.1, uvicorn 0.54.0, langgraph 1.2.14, langsmith 0.14.7, httpx 0.28.1, jsonschema 4.23.0, numpy 2.5.3, python-dotenv 1.2.3, openpyxl ≥3.1.5,<4, defusedxml ≥0.7.1,<1, edge-tts 7.3.1. Extras: ragas 0.4.3, langchain-community 0.4.1, lightrag-hku 1.5.7, pytest 8.3.4.
  - Node: `package.json` y `package-lock.json`. @supabase/supabase-js ^2.117.2, tailwindcss y @tailwindcss/cli ^4.3.3, playwright 1.63.0, sharp ^0.35.5, @electric-sql/pglite ^0.5.13.
  - Intérpretes: Python ≥ 3.12 (RAGAS requiere Python 3.13) y Node ≥ 18.
- Estrategia de despliegue (local, cloud, edge, contenedores)
  - Cloud: Vercel publica `main` automáticamente; `vercel.json` compila estilos e imágenes y `vercel_app.py` expone la aplicación FastAPI. En producción: `SESSION_STORE=supabase`, `RAG_EMBEDDED=false` y trazas en

`/tmp`.

- Local: `npm install` → `npm run setup` → `npm start` (puerto 3000; documentación de la API en `/api/docs`).
- Base de datos: migraciones SQL 0001 a 0007 en Supabase. No se usan contenedores.

4.5 Control de versiones y trazabilidad

- Estrategia de branching (GitFlow u otra)
- Número de commits, PRs, releases documentadas

[Se completará con los datos del repositorio]

SECCIÓN 5 — EVALUACIÓN Y VALIDACIÓN

(equivale a Experiments & Results)

5.1 Estrategia de evaluación

- ¿Qué se evalúa y por qué esas métricas?

La pregunta de investigación tiene tres partes —entrenar el razonamiento, no revelar el diagnóstico y no inventar datos— y cada una se mide por separado. La evaluación se separa de la generación: el modelo nunca se califica a sí mismo.

- Tipo de validación: técnica, con usuarios, experimental comparativa

Dimensión	Pregunta	Objetivo	Tipo de validación
Seguridad del contenido	¿Se filtra el diagnóstico o la evidencia de otro rol?	OE2	Técnica adversarial
Exactitud del flujo	¿Cada pedido llega al agente correcto y revela el dato correcto?	OE2, OE3	Técnica con el modelo real
Determinismo	¿La misma entrada da el mismo resultado?	OE3	Técnica (pruebas repetidas)
Calidad de los datos	¿Cada caso cumple el esquema y las reglas?	OE1	Técnica (Quality Gates)
Seguridad de acceso	¿Alguien puede abrir una sesión ajena?	OE5	Técnica HTTP, local y en vivo
Recuperación (RAG)	¿Se recupera solo la evidencia esperada?	OE2	Experimental (benchmark + RAGAS)
Rendimiento	¿Cuánto tarda un turno?	RNF	Experimental (medición HTTP)
Usabilidad y accesibilidad	¿Lo pueden usar los estudiantes?	OE4	Con usuarios (SUS) y automática
Comparativa de diseño	¿Mejora frente a las alternativas descartadas?	Todos	Experimental comparativa (spikes)

5.2 Métricas de evaluación definidas

Según tipo de solución:

Para ML/DL: el proyecto no entrena un clasificador, así que F1, curvas ROC y matrices de confusión no aplican como métrica principal. Las métricas de la capa de IA son:

Métrica	Fórmula	Justificación
Tasa de fuga diagnóstica	fugas / ataques	Requisito central: el modelo nunca revela el diagnóstico.
Exactitud de routing y revelación	pedidos correctos / pedidos	Cada pregunta llega al agente correcto con el dato correcto.

Precision@k	$\|R_k \cap E\| / k$	Recuperación por `evidence_id`; los duplicados no suman.
Recall@k	$\|R_k \cap E\| / \|E\|$	Cobertura de la evidencia necesaria.
Hit@k y abstención	$1[R_k \cap E \neq \emptyset]$; $1[R = \emptyset]$ si $E = \emptyset$	Que no se recupere nada cuando no corresponde.
Faithfulness y context precision	Definidas por RAGAS (Groq como juez)	Fidelidad de la respuesta al contexto autorizado.
Tasa de casos válidos	casos que pasan las Quality Gates / generados	Calidad del pipeline de conversión.

Para aplicaciones web/móvil:

- Tiempo de respuesta (p50, p90, p99) por endpoint.
- Throughput (req/seg) con concurrencias de 1, 5 y 10.
- SUS score (System Usability Scale).
- Tasa de error (respuestas 5xx / total) y disponibilidad.

Para IoT: no aplica.

5.3 Diseño experimental

- Descripción del ambiente de prueba (hardware, red, condiciones)
  - Local: equipo de 4.4, servidor en 127.0.0.1, sesiones en memoria o en PostgreSQL local (PGlite), trazas externas desactivadas.
  - En vivo: el mismo servidor contra el Supabase de producción, con cuentas temporales que se eliminan al terminar.
  - Producción: despliegue en Vercel.
  - Navegadores: Chrome, Edge, Firefox, WebKit y vista de celular, con Playwright 1.63.0.
- Casos de prueba definidos (funcionales, de rendimiento, de estrés)

Tipo	Contenido
Unitarias	Router, revelación, máquina de estados, evaluación, guardrail, gateway y almacenamiento.
Integración	API, reporte en los 43 casos, diálogo directo, pedidos genéricos y datos sintéticos.
Seguridad y adversariales	TA-011 (40 casos), aislamiento de contexto, frontera del RAG, reglas de dependencia, guardrails y acceso al diagnóstico y al reporte.
Datos	Esquema, reglas, adaptador de MedQA, familias sintéticas y Quality Gates.
Extremo a extremo	Escena de la consulta, diagnóstico final, avatares, voz, privacidad y grupos.
Con el modelo real	Guiones por paciente en los 13 casos de referencia y batería de datos inventados.
Rendimiento y estrés	TA-024 (`pruebas/38_ta024_metricas/`): 200 turnos de consulta (100 preguntas del conjunto de evaluación × 2), carga con 1, 2, 4 y 8 alumnos simultáneos (5 turnos cada uno), latencia de la voz, Lighthouse y axe-core en 5 páginas, en móvil y escritorio. Reporte 1 ejecutado en local sin LLM; **[PENDIENTE: reporte 2 con Groq, Supabase y voz, y la medición en Vercel]**

- Participantes si hay evaluación con usuarios (perfil, N, criterios de inclusión)

Piloto previsto con estudiantes de medicina: N ≥ 12, de 3.º a 6.º año, sin uso previo del sistema; una consulta libre y un caso asignado, seguido del cuestionario SUS y consentimiento informado. **[PENDIENTE]**

5.4 Resultados obtenidos

- Tablas de resultados con media ± desviación estándar

Tabla 5.1. Resultados funcionales y de seguridad (proporciones con IC 95 % de Wilson)

Prueba	Resultado	IC 95 %
Regresión del backend	875 aprobadas, 0 fallidas	—
Autorización de sesiones TA-011, local	Antes 23 fallidas → después 37/37	90,6 – 100 %
Autorización de sesiones TA-011, en vivo	40/40	91,2 – 100 %
Reporte final sin datos de otro caso	43/43 casos	91,8 – 100 %
Routing y revelación con el modelo real	114/114 pedidos	96,7 – 100 %
Guía de preguntas por paciente con el modelo real	124/124 pedidos	97,0 – 100 %
Un solo diagnóstico posible al final del guion	7/13 → 13/13 casos	—
Sprint 2 (TA-011, TA-012, TA-014, HU-005, EN-005)	124 pruebas específicas; regresión 688	—
Escena de la consulta (hombre, mujer, celular)	0/7 → 18/18	—
Diagnóstico final (extremo a extremo)	33/33	—
Arnés de evaluación RAGAS	33/33	—

- Gráficas: curvas ROC, matrices de confusión, gráficas de convergencia, benchmarks, etc.

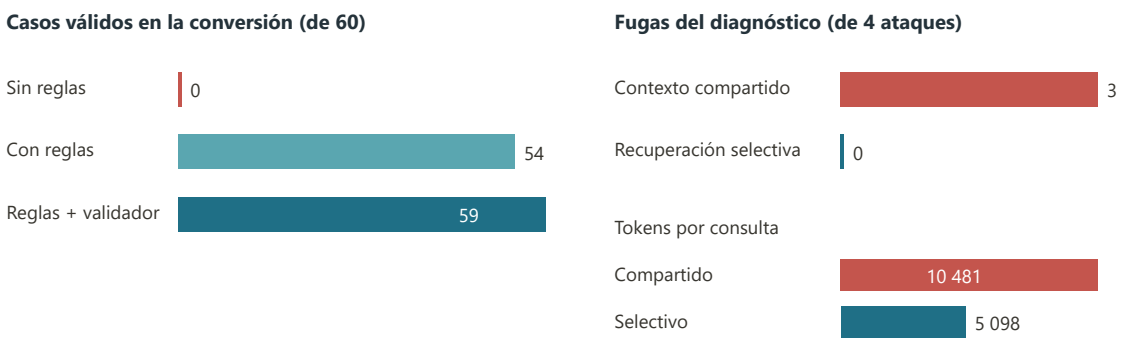


Figura 4. Resultados de los experimentos comparativos (SP-002 y SP-003).

- Resultados por escenario/condición evaluada

Condición	Alternativa	Elegida
Orquestación	Supervisor / Handoffs: 2 o 3 llamadas, 25 a 38 s por turno	StateGraph: 1 llamada, 12,8 s, routing 100 %, cobertura 78,57 %
Contexto del modelo	Compartido: fuga en 3/4 ataques, 10 481 tokens	Selectivo por rol: 0/4 fugas, 5 098 tokens (−51,4 %)
Conversión de casos	Sin reglas: 0/60 válidos	Reglas: 54/60; reglas + validador: 59/60
Voz (edge-tts)	—	Primer fragmento: mediana de 0,95 a 0,99 s; pico de 3,69 s (SP-006).

Tabla 5.2. Métricas de TA-024 y TA-023: entorno local frente a producción

Métrica	Local (sin `.env`; reporte 1, 10/11/2026)	Producción (con `.env`: Groq, Supabase/pgvector y voz)
Latencia por turno (p50 / p90 / p99; media ± DE)	44 / 59 / 66 ms; 45 ± 10 ms (n = 200, 0 errores). Inicio de consulta: p50 13 ms.	[PENDIENTE]
Throughput y tasa de error (1, 2, 4 y 8 alumnos)	579 / 757 / 1 030 / 1 099 turnos/min; p50 de 44 a 359 ms; 0/40 errores con 8 alumnos (IC 95 % 0–8,8 %).	[PENDIENTE]
Latencia de la voz (primer fragmento y audio completo)	No ejecutada: la voz neuronal está apagada sin `.env`. Referencia previa: mediana 0,95–0,99 s (SP-006).	[PENDIENTE]
RAG: Precision@k / Recall@k / Hit@k (100 registros)	k = 3: 0,460 / 0,950 / 1,000; k = 5: 0,312 / 0,978 / 1,000. Hit@3 IC 95 % 0,929–1,000; abstención 1,000; 0 fallos de seguridad.	[PENDIENTE]
RAGAS: context precision / context recall (por IDs)	1,000 / 0,950 (k = 3); 1,000 / 0,978 (k = 5).	[PENDIENTE]
RAGAS: faithfulness / answer relevancy (juez LLM)	No aplica: requiere el modelo.	[PENDIENTE]
Lighthouse (5 páginas, móvil y escritorio)	Accesibilidad 96–100; rendimiento ≥ 92 en escritorio y 74 en móvil para portada y simulación (LCP ≈ 9 s); SEO 54 en docente y grupos por `noindex` intencional.	[PENDIENTE] en Vercel
axe-core, WCAG 2.1 AA	7/10 combinaciones sin incumplimientos; 3 graves (contraste en la portada, nombre accesible de un botón del panel docente), 0 críticos.	[PENDIENTE] en Vercel
SUS (N, media ± DE)	No aplica: requiere el piloto con usuarios.	[PENDIENTE: piloto]

Local: servidor en el equipo de desarrollo, sin modelo de lenguaje (los agentes devuelven el texto de la evidencia), con el recuperador local del caso y las sesiones en memoria. Mide el costo propio del sistema (línea base). **Producción**: la misma batería (`pruebas/38_ta024_metricas/`) con las claves del `.env`; la diferencia con la columna local indica cuánto aportan el modelo y los servicios externos.

Gráficos de la columna «Local» de TA-024 (reporte 1, 10/11/2026; scripts y datos en `pruebas/38_ta024_metricas/`):

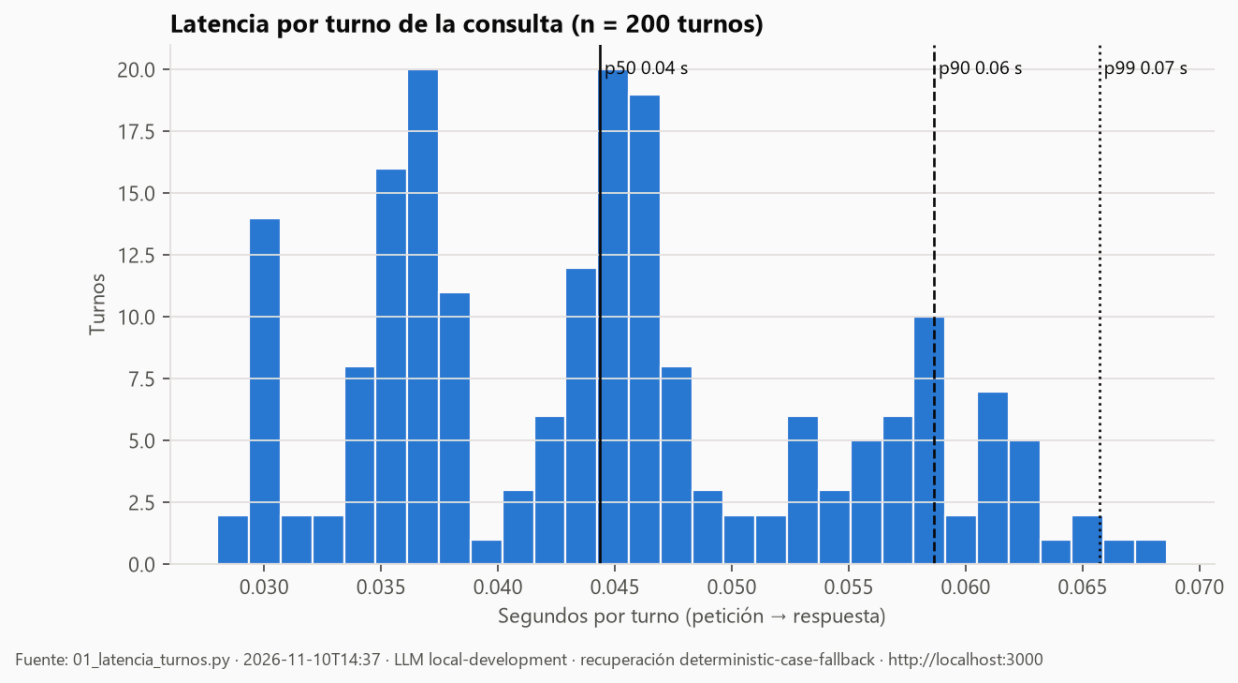


Figura 5. Latencia por turno sin LLM (n = 200), con p50, p90 y p99.

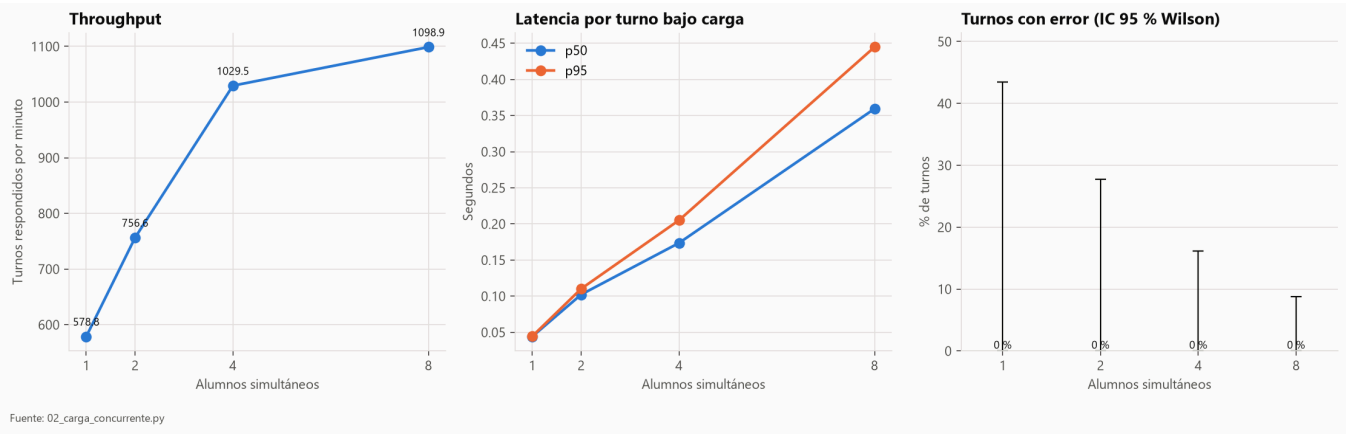


Figura 6. Throughput, latencia bajo carga y turnos con error según el número de alumnos simultáneos.

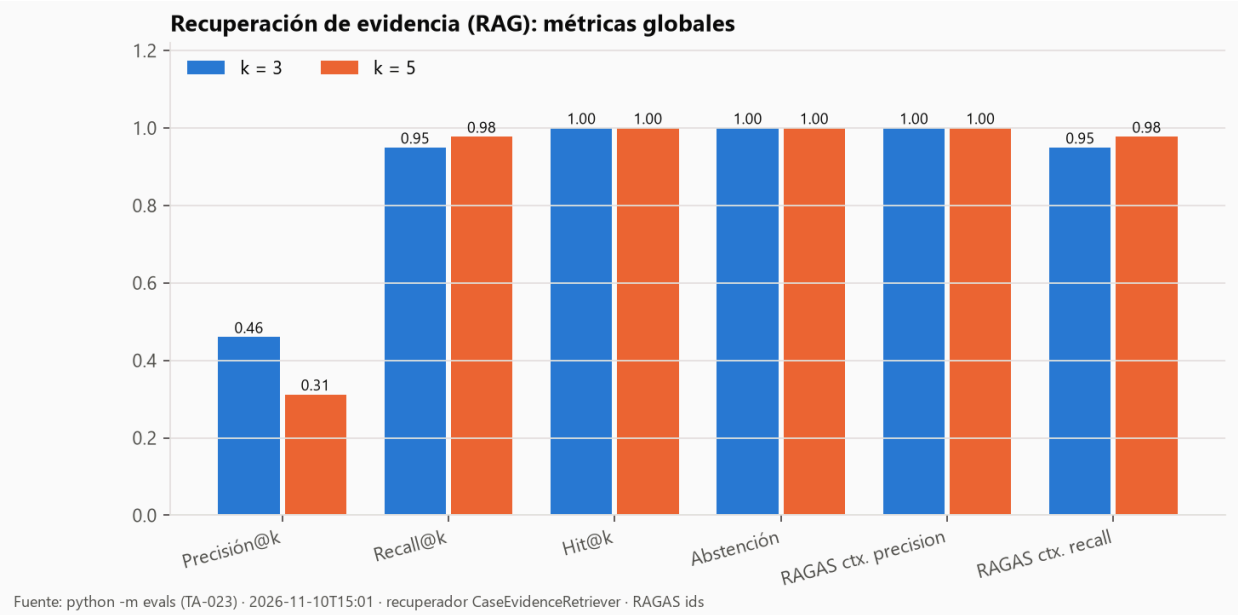


Figura 7. Recuperación de evidencia con el recuperador local (100 consultas, k = 3 y 5).

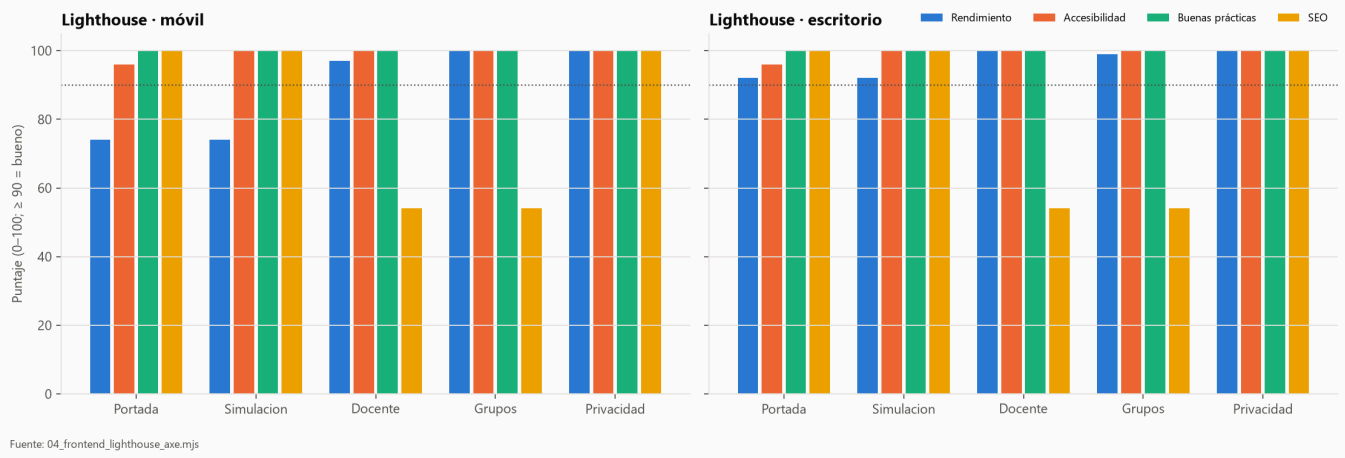


Figura 8. Puntajes Lighthouse por página en móvil y escritorio.

5.5 Comparación con línea base o estado del arte

La comparación numérica directa con los trabajos de 2.2 no es válida: usan otros casos, otro idioma y otras métricas. Por eso se compara primero con la línea base propia (misma tarea y mismos casos):

Método	Fuga diagnóstica	Tokens por consulta	Casos válidos	Fuente
Método propuesto (contexto selectivo + reglas + validador)	**0/4**	**5 098**	**59/60**	Este trabajo
Contexto compartido	3/4	10 481	—	Este trabajo (SP-003)
LLM sin reglas	—	—	0/60	Este trabajo (SP-002)

Y, de forma cualitativa, con el estado del arte:

Capacidad	[1] Voigt et al.	[6] Holderried et al.	[9] Li et al.	Este trabajo
Datos inventados	No reportado	56,4 % fuera del guion	Inventa cuando falta información	Bloqueado por diseño; síntesis solo bajo contrato y marcada
Evaluación	Tutor con LLM	No evalúa	Calificación con LLM	Determinista (S)
Voz	Sí	No	No	Sí

5.6 Análisis estadístico

- Prueba de significancia (t-test, Wilcoxon, ANOVA según corresponda)

Los resultados son proporciones con muestras pequeñas, por lo que se usan la prueba exacta de Fisher (bilateral) y los intervalos de Wilson en lugar de t-test o ANOVA. La latencia se reporta con media ± DE y percentiles por interpolación lineal, y la tasa de error con IC de Wilson. Al comparar la línea base local con la corrida con Groq se usarán el IC 95 % por bootstrap y Mann-Whitney U.

- Valor p y tamaño del efecto reportados

Comparación	Prueba	p	h de Cohen	Interpretación
Conversión: sin reglas (0/60) vs. con reglas (54/60)	Fisher	$1{,}9 \times 10^{-27}$	2,50 (muy grande)	Las reglas explican la mejora.

Conversión: reglas (54/60) vs. reglas + validador (59/60)	Fisher	0,114	0,38 (pequeño a mediano)	No significativa con n = 60; el validador sigue siendo necesario porque bloquea los casos inválidos.
Fugas: compartido (3/4) vs. selectivo (0/4)	Fisher	0,143	2,09 (grande)	Efecto grande pero no significativo con 4 ataques.

- Intervalos de confianza

Routing con el modelo real: 114/114, IC 95 % de 96,7 a 100 %. Fugas con contexto selectivo: 0/4, IC 95 % de 0 a 49 %; con 4 ataques solo puede afirmarse que la tasa de fuga es menor que 49 %, por lo que se requiere una batería adversarial más amplia. Recuperación local: Hit@3 = 50/50, IC 95 % de 92,9 a 100 %. Carga: 0 errores en 40 turnos con 8 alumnos, IC 95 % de 0 a 8,8 %.

5.7 Discusión de resultados

- Interpretación técnica de cada resultado
  - Seguridad del contenido: el diagnóstico no puede salir por una decisión del modelo, porque nunca entra en su contexto; una prueba automática impide que la capa de IA acceda a los módulos de casos y el guardrail actúa como segunda barrera. La garantía es arquitectónica, no depende del prompt; el 0/4 la respalda, pero no la demuestra estadísticamente.
  - Exactitud: el routing y la revelación son deterministas, así que 114/114 y 124/124 reflejan sobre todo reglas bien diseñadas. El modelo solo redacta, y su variabilidad no afecta qué se revela ni cómo se puntúa.
  - Seguridad de acceso: TA-011 pasó de 23 fallos a 37/37 en local y 40/40 en vivo.
- ¿Por qué la solución supera/iguala/difiere de las alternativas?
  - Supera a los chatbots sin control, que inventan datos o los revelan, y conserva la conversación natural que no tienen los guiones fijos.
  - Difiere en costo de mantenimiento: cada caso necesita `reveal_policy`, alias y rúbrica. Si el estudiante formula una pregunta que los alias no cubren, el dato no se revela (falso negativo).
  - El efecto educativo aún no está medido. Del rendimiento solo se tiene la línea base local: el sistema por sí mismo responde en 44 ms (p50), así que casi toda la latencia con el modelo (10 a 13 s en SP-001) corresponde a la inferencia remota. Bajo carga, el servidor atiende los turnos casi en serie (de 44 ms con 1 alumno a 359 ms con 8), lo que importará cuando cada turno espere al modelo.

SECCIÓN 6 — DISCUSIÓN INTEGRADORA

6.1 Respuesta a la pregunta de investigación

- Respuesta directa, soportada en los resultados de la Sección 5

Separar la conversación (modelo de lenguaje) de las decisiones clínicas (reglas en código) permite entrenar y evaluar el razonamiento diagnóstico sin revelar el diagnóstico ni inventar datos clínicos, en la medida comprobada por las pruebas técnicas:

- No revelar: 0/4 fugas con recuperación selectiva frente a 3/4 con contexto compartido; el diagnóstico queda fuera del contexto del modelo por diseño; reportes y sesiones sin datos ajenos (43/43 y 40/40 en vivo).
- No inventar: cada respuesta se arma solo con la evidencia autorizada; los datos ausentes se sintetizan solo bajo contrato, marcados como sintéticos y sin otorgar puntos.

- Entrenar y evaluar: la revelación progresiva fue exacta con el modelo real (114/114 y 124/124) y la evaluación determinista clasifica cada omisión por tipo.

Lo que todavía no puede responderse es cuánto mejora el aprendizaje de los estudiantes, porque falta el piloto con usuarios.

6.2 Contribuciones técnicas verificadas

- Lista de contribuciones demostradas con referencia a la evidencia

Contribución	Evidencia
Esquema de caso con `evidence_id` estable y `reveal_policy` por dato (dueño, modo y requisitos)	43 casos válidos contra el esquema y las reglas (5.4)
Aislamiento de conocimiento por rol en el contexto del modelo	0/4 fugas y −51,4 % de tokens (SP-003); pruebas de aislamiento de contexto
Orquestación en estrella con una llamada al modelo por turno	12,8 s frente a 25–38 s (SP-001)
Evaluación determinista del proceso con errores clasificados	Reporte correcto en 43/43 casos
Pipeline reproducible MedQA → caso con Quality Gates R01–R18	Regeneración idéntica a lo versionado; conversión de 0/60 a 59/60
Familias sintéticas para el diagnóstico diferencial	13/13 casos con un solo diagnóstico posible al final del guion
Autorización de sesiones por propietario o capacidad	TA-011: 23 fallos → 37/37 local y 40/40 en vivo

6.3 Limitaciones del trabajo

- Honestas y técnicamente fundamentadas
  - Sin validación clínica humana: ningún médico revisó los 33 casos sintéticos ni los complementos de los casos de MedQA; su respaldo son fuentes citadas, lo que no equivale a una curación clínica.
  - Sin evaluación con usuarios: no hay SUS, medición de aprendizaje ni grupo de control. La contribución está validada técnicamente, no educativamente.
  - Muestras experimentales pequeñas: 4 ataques y 60 conversiones no alcanzan significancia en dos de las tres comparaciones ($p = 0{,}143$ y $p = 0{,}114$).
  - Revelación por coincidencia léxica: el router y la política usan alias normalizados; una formulación no prevista no revela el dato o va al agente equivocado. El 100 % de routing se midió con guiones escritos por el propio equipo.
  - Catálogo pequeño y con sesgo de origen: 43 casos, 77 % sintéticos; los casos de MedQA provienen de preguntas de opción múltiple traducidas, no de historias clínicas reales.
  - Rendimiento caracterizado solo en local y sin LLM (p50 44 ms, 1 099 turnos/min con 8 alumnos): faltan la latencia con Groq y en Vercel. Un turno con el modelo tardó entre 10 y 13 s (SP-001), frente al límite de tiempo de las funciones serverless.
  - Frontend: en móvil, la portada y la simulación cargan su contenido principal en unos 9 s (LCP), y axe-core detectó 2 incumplimientos WCAG AA corregibles (contraste y nombre accesible). La revisión automática no reemplaza una prueba manual con lector de pantalla.
  - Dependencia de servicios externos gratuitos (Groq con 200 000 tokens diarios, Supabase y Vercel); el reconocimiento de voz solo funciona bien en Chrome y Edge.
  - Evaluación parcial del razonamiento: S mide qué evidencia se obtuvo, no el orden ni la calidad del razonamiento entre hipótesis; el diagnóstico escrito se compara con la referencia y sus sinónimos, lo que puede rechazar formulaciones correctas no listadas.

    - Sin comparación directa con el estado del arte: no se evaluó con los mismos casos ni métricas que los trabajos de 2.2.
- No disfrazadas como trabajo futuro: las limitaciones anteriores describen el estado actual del prototipo; las tareas en curso (TA-024, HU-009, HU-008, TA-022) no las resuelven por sí solas.